



Pashov Audit Group

LiFi - July Security Review

July 21st 2026 - July 26th 2026



Executive Summary

A time-boxed security review of LiFi - July was conducted through the Pashov Audit Group bug-hunt program. This report documents 4 issues across the reviewed codebase, including findings reported by Slayer Security.

Project	LiFi - July
Timeline	July 21st 2026 - July 26th 2026

About

Pashov Audit Group is a security research collective focused on smart-contract and protocol security. This report documents the issues surfaced during a time-boxed review delivered through our competitive bug-hunt program.

METHODOLOGY

Researchers independently reviewed the in-scope code and reported the issues they found. Each submission was then triaged and validated by the judging team; duplicates were merged and severities were finalized. Every accepted issue appears below with its impact and a recommended remediation. The client's response to each issue was then reviewed, and its resolution status - fixed, acknowledged, or invalidated - is recorded with the finding.

DISCLAIMER

A security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where the researchers try to find as many issues as possible. We can not guarantee 100% security after the review. Subsequent reviews, bug bounty programs and on-chain monitoring are strongly recommended. The client's remediation of each issue was reviewed after the hunt; the outcome is recorded in the finding's Resolution section.

Risk Classification

Critical

Loss or compromise of critical assets, or protocol-wide impact. Must be fixed before launch.

High

Significant material loss of assets, or serious harm to a group of users.

Medium

Moderate material loss, or moderate harm, often under conditional assumptions.

Low

Minor impact, limited scope, or requires unlikely conditions to trigger.

Info

No direct security impact - best-practice, code-quality or hardening suggestion.

Scope

<https://github.com/catalystsystem/lifi-intent/tree/cf2058b6fa96f2ef507eb99c1773a0031367da9d>

commit: cf2058b6fa96f2ef507eb99c1773a0031367da9d

- src/input/InputSettlerBase.sol



- src/input/escrow/InputSettlerEscrow.sol
- src/input/escrow/InputSettlerEscrowLIFI.sol
- src/integrations/oracles/polymer/PolymerOracle.sol
- src/integrations/oracles/polymer/external/interfaces/ICrossL2ProverV2.sol
- src/interfaces/IInputSettlerEscrow.sol
- src/libs/MandateOutputEncodingLib.sol
- src/output/OutputSettlerBase.sol

Findings

SEVERITY	COUNT
Low	4
Total	4

ID	TITLE	SEVERITY	STATUS
L-01	cUSDCv3 maximum-amount semantics allow an underfunded order to drain InputSettlerEscrowLIFI	Low	Acknowledged
L-02	ERC-3009 authorization cross-use allows atomic orders to be front-run and converted into long-lived escrow deposits	Low	Acknowledged
L-03	Latest-fill timestamp extends purchase eligibility and enables underpriced escrow takeover	Low	Acknowledged
L-04	LI.FI callbacks receive gross inputs after fee deduction	Low	Fixed

Low Severity

[L-01] cUSDCv3 maximum-amount semantics allow an underfunded order to drain InputSettlerEscrowLIFI

Low

Acknowledged

SUMMARY

`InputSettlerEscrowLIFI` explicitly states that it does not support fee-on-transfer tokens.



```
/**
 * @title LIFI Input Settler supporting an explicit escrow
 * @notice This contract is implemented as an extension of the OIF
 * It inherits all of the functionality of InputSettlerEscrow but adds same chain swaps and
 * governance fee.
 *
 * This contract does not support fee on transfer tokens.
 *
 * The ownable component of the smart contract is only used for fees.
 */
```

However, cUSDCv3 is not a fee-on-transfer token, so users and solvers are free to interact/open/refund/finalise with these tokens. cUSDCv3 contains a special case for `amount == type(uint256).max` in its transfer function that results in only the user's balance being transferred.

`InputSettlerEscrowLIFI` inherits the collection logic from `InputSettlerEscrow`. Both inherited `_open()` and `_openForWithPermit2()` accept a successful token transfer without verifying the escrow contract's actual balance increase.

Consequently, an attacker can create an order whose nominal cUSDCv3 input is `type(uint256).max` while transferring only their small current balance. The order is nevertheless marked as fully deposited. When the order is later refunded, `InputSettlerEscrowLIFI` transfers the nominal `type(uint256).max` amount. Under cUSDCv3's semantics, this transfers the contract's entire cUSDCv3 balance, including funds belonging to other orders.

The Permit2 collection path is also affected because Permit2 provides authorization but does not guarantee that the token moved exactly the requested amount.

IMPACT

An attacker can steal the complete cUSDCv3 balance held by `InputSettlerEscrowLIFI`.

Funds deposited for unrelated orders are commingled in the contract. Therefore, the attack does not merely underfund the attacker's order: its refund consumes cUSDCv3 backing other users' outstanding orders.

After the attack:

- the attacker receives the contract's entire cUSDCv3 balance;
- unrelated orders remain marked as deposited but are no longer backed;
- subsequent refunds and finalisations revert because the contract lacks sufficient funds.

Although `InputSettlerEscrowLIFI` documents fee-on-transfer tokens as unsupported, this exclusion does not cover cUSDCv3's maximum-amount transfer behaviour.

RECOMMENDATION

For every direct and Permit2 token collection, compare `InputSettlerEscrowLIFI`'s balance before and after the transfer and require the increase to equal the requested amount.

For Permit2 batches, aggregate expected amounts by unique token before validating the resulting balance deltas. Also consider rejecting `type(uint256).max` inputs and enforcing a supported-token allowlist.



RESOLUTION - ACKNOWLEDGED

Client response (LI.FI): "Ack. This is a small class of tokens and we would rather document it than introduce additional gas overhead for these tokens."

[L-02] ERC-3009 authorization cross-use allows atomic orders to be front-run and converted into long-lived escrow deposits

Low

Acknowledged

SUMMARY

ERC-3009 warns that when multiple contract functions accept receive authorizations, applications should domain-separate their nonces to prevent an authorization intended for one function from being used through another. See the [ERC-3009 Security Considerations](#).

Additionally, if there are multiple contract functions accepting receive authorizations, the app developer could dedicate some leading bytes of the nonce as an identifier to prevent cross-use.

`InputSettlerEscrowLIFI` exposes two functions that accept the same ERC-3009 authorization:

- inherited `openFor()`
- `openForAndFinalise()`

Both functions call `_openForWithAuthorization()` using the same order, sponsor, signature, and `orderId`. This produces identical ERC-3009 authorization parameters:

```
from      = sponsor
to        = InputSettlerEscrowLIFI
value     = input amount
validAfter = 0
validBefore = order.fillDeadline
nonce     = orderId
```

Consequently, an authorization intended for the atomic `openForAndFinalise()` operation can be front-run and consumed through `openFor()`. The sponsor's funds are deposited into escrow and the order is marked `Deposited`, but the intended atomic transfer, callback, and fill validation never occur.

IMPACT

An attacker can convert an intended atomic `openForAndFinalise()` operation into an ordinary escrow deposit. The sponsor's funds remain locked until the order is independently settled, proven unfilled, or expires.

Because `orderFinalised()` may execute arbitrary callback logic, the attack can also prevent time-sensitive actions such as debt repayment, collateral top-ups, option exercise, auction participation, multi-hop swaps, or batch settlement of dependent orders. This may cause liquidation penalties, lost opportunities, or failed settlements even if the escrowed assets are eventually refunded.

Also, the attack is systematically repeatable across newly broadcast `openForAndFinalise()` transactions: each fresh order and ERC-3009 authorization can be front-run through `openFor()` at the attacker's gas cost. Thus, leading to DOS as well.



Orders with distant deadlines can experience a long-lived fund lock. The attacker cannot directly withdraw the funds but can repeatedly disrupt public atomic transactions at little cost.

RECOMMENDATION

Use function-specific ERC-3009 nonces so that authorisations cannot be used across entry points. For example:

```
keccak256(abi.encode("OIF_ERC3009_OPEN", orderId, inputIndex))
```

and:

```
keccak256(abi.encode("OIF_ERC3009_ATOMIC", orderId, inputIndex))
```

Pass the intended operation into `_openForWithAuthorization()` and derive the nonce from both the operation and order. The Permit2 witness should similarly bind whether the signature authorises `openFor()` or `openForAndFinalise()`.

RESOLUTION - ACKNOWLEDGED

Client response (Li.FI): "The final transaction should be cheaper for the solver everything considered. The final decision is acknowledged. This is within expected behaviour."

[L-03] Latest-fill timestamp extends purchase eligibility and enables underpriced escrow takeover

Low

Acknowledged

LOCATION

InputSettlerPurchase:_purchaseGetOrderOwner:83-102

ROOT CAUSE

The intended behavior is that a purchaser may acquire a solver's completed-order claim only within the solver-authorized purchase window, while paying the consideration specified by the signed purchase quote. Once that window has elapsed, the solver should remain the effective owner of the escrow claim, and the purchase price should not disappear through per-row rounding.

The actual code records a purchaser without checking fill timing and later validates eligibility using `_maxTimestamp(solveParams)`. A purchase made after the first fill's window can therefore be accepted when a later output has a sufficiently recent fill. At the same time, `_purchaseOrder` floors every input row independently:

```
uint256 amountAfterDiscount =  
(allocatedAmount * (DISCOUNT_DENOM - discount)) / DISCOUNT_DENOM;  
_transferFromSender(tokenId, newDestination, amountAfterDiscount);
```

This can make all consideration transfers zero while preserving the purchaser record. Finalisation then treats that purchaser as the owner and releases the full escrow to the purchaser-selected destination, creating a material mismatch between consideration paid and value claimed.



IMPACT

A holder of a valid solver-signed purchase quote can acquire purchaser ownership for zero or negligible consideration when the quote's discount and fragmented input amounts cause every discounted leg to round down. By timing the purchase before a later output fill, the purchaser can pass the flawed latest-fill check and receive the entire origin escrow after all outputs are filled.

EXPLOIT PATH

1. A solver signs an `OrderPurchase` quote with a high discount and `timeToBuy`, and the deposited order contains multiple small input rows such that each discounted row rounds down to zero.
2. After the first output is filled and the first-fill purchase window expires, the attacker submits `purchaseOrder` before a later output is filled; the order is still `Deposited` and has not expired, so the purchase record is stored.
3. A later output is filled before the order's fill deadline, producing a maximum fill timestamp that is newer than the stored purchase threshold.
4. The attacker finalises using the purchaser identity and destination. `_purchaseGetOrderOwner` returns the purchaser, `_resolveLock` transfers the complete escrow, and the solver receives no or negligible purchase consideration.

RECOMMENDATION

Measure eligibility using the earliest fill timestamp when that is the intended purchase-window anchor:

```
uint256 orderTimestamp = _minTimestamp(solveParams);
delete purchaseDetails.lastOrderTimestamp;
delete purchaseDetails.purchaser;
if (lastOrderTimestamp <= orderTimestamp) return purchaser;
return solver;
```

Prevent nonzero input rows from silently becoming free, or bind an explicit aggregate consideration into the signed quote:

```
uint256 amountAfterDiscount =
(allocatedAmount * (DISCOUNT_DENOM - discount)) / DISCOUNT_DENOM;
if (allocatedAmount != 0 && amountAfterDiscount == 0) revert InvalidPurchaseAmount();
_transferFromSender(tokenId, newDestination, amountAfterDiscount);
```

RESOLUTION - ACKNOWLEDGED

Client response (LI.FI): "Ack. Purchase can compute this value-off chain to avoid. For a 0-result, the discount have to be very large or allocatedAmount has to be very small. In either case, the impact is very low."

[L-04] LI.FI callbacks receive gross inputs after fee deduction

Low

Fixed



LOCATION

InputSettlerEscrowLIFI:_resolveLock/finalise:142-164,215-249

ROOT CAUSE

The intended callback behavior is to describe the assets made available to the destination after settlement. In the base escrow flow, the input array corresponds directly to the amounts transferred to the destination before `orderFinalised` is invoked.

The LI.FI override deducts governance fees in `_resolveLock`, sending the fee to the owner and only the remainder to the destination, but the finalisation functions still pass the unchanged gross `order.inputs` array:

```
_finalise(order, orderId, solveParams[0].solver, destination);
if (call.length > 0) {
  IInputCallback(destination.fromIdentifier()).orderFinalised(order.inputs, call);
}
```

As a result, callback recipients receive arguments that overstate their newly delivered balances. Integrations that use those values for transfers, swaps, or validation can fail or execute against incorrect amounts whenever governance fees are enabled.

IMPACT

A callback destination can receive an amount lower than the amount specified in its callback arguments. Integrations that trust the callback amounts can revert, fail to forward the expected funds, or execute incorrect downstream accounting, disrupting otherwise valid fee-enabled settlements.

EXPLOIT PATH

1. Open an LI.FI order with one 100-unit ERC-20 input and a callback destination.
2. Set the governance fee to 1% and finalise the order through `finalise` or `finaliseWithSignature`.
3. `_resolveLock` transfers 1 unit to the governance owner and 99 units to the callback destination.
4. `orderFinalised` receives an input amount of 100, so a callback attempting to use all 100 units reverts or miscalculates its downstream operation.

ADDITIONAL CONTEXT

- The mismatch exists across both the normal LI.FI finalise path and the fast-path callback flow wherever governance fees are enabled.
- Downstream integrations may over-credit users or revert when they trust callback parameters as delivered balances.

RECOMMENDATION

Construct callback inputs using the net amounts actually sent to the destination:



```
uint256[2][] memory callbackInputs = new uint256[2][](inputs.length);
for (uint256 i; i < inputs.length; ++i) {
    uint256 feeAmount = _calcFee(inputs[i][1], fee);
    uint256 netAmount = inputs[i][1] - feeAmount;
    callbackInputs[i] = [inputs[i][0], netAmount];
}
IInputCallback(destination).orderFinalised(callbackInputs, call);
```

Alternatively, change the callback interface or LI.FI documentation to explicitly identify amounts as gross and require callbacks to calculate the fee independently. The adjusted-amount approach is preferable because it preserves the base interface's intuitive delivered-asset semantics.

RESOLUTION - FIXED

Client response (LI.FI): Fixed: [catalystsystem/li-fi-intent@1308c4b](https://github.com/catalystsystem/li-fi-intent/pull/1308c4b)